AULA 2

Aumentando seu repertório

Continuando...

Na semana passada, desenhamos a arquitetura do nosso sistema e criamos o primeiro fluxo no Flowise.

Agora chegou o momento de dar uma "cara" para o nosso projeto. Afinal, de que adianta uma IA brilhante se o usuário não consegue interagir com ela?

Nesta semana, vamos construir interfaces *web* modernas para ver seu *chatbot* ganhar vida. Vamos lá?

O problema da tela preta

Se você já tentou mostrar um projeto de programação para alguém que não é da área e a única coisa que você tinha era um terminal com texto verde em fundo preto, provavelmente a reação foi algo como "Legal... mas o que isso faz?".

Isso é o que eu chamo de **problema da tela preta**.

Na nossa cabeça de desenvolvedores, o terminal (isto é: a tela preta) é uma ferramenta sensacional. Mas para 99% das pessoas, é algo que causa medo e confusão.

No mercado de trabalho, isso tem uma consequência prática muito importante: **quem paga o seu salário (ou financia a sua *startup*) não aprova um projeto que eles não conseguem visualizar**. Se você quer que seu projeto de IA seja aprovado, financiado e utilizado, ele precisa de uma interface que qualquer pessoa consiga entender.

#FICADICA

Algo que você consegue demonstrar só em uma tela preta do terminal, ou em um documento do Word ou uma apresentação do PowerPoint não é um produto.

Se seu sistema só roda em um terminal local, ele tem valor limitado.

A interface é o que transforma código em produto.

E por que dar atenção a isso?

Antes de mergulhar no código, quero mostrar por que essa semana pode ser uma das mais valiosas para sua produtividade.

Afinal, quem sabe criar interfaces rápidas para seus sistemas ganha autonomia. Em vez de esperar meses pelo time de *front-end*, você mostra um protótipo funcional em horas.

EXPERIÊNCIAS QUE INSPIRAM

Nas empresas em que trabalhei eu conseguia aprovar projetos mais rápido do que os meus colegas porque criava interfaces visuais nas quais as pessoas podiam clicar, mexer e dar opiniões.

Pense comigo: uma interface de *chat* funcional poderia levar dias para ser construída, e você precisaria de alguns meses para estudar todos os tópicos necessários para conseguir criar alguma coisa. Agora, podemos fazer isso em alguns minutos ou em algumas poucas horas. Isso também vale para aplicativos que você pode compartilhar, ou formulários com lógicas específicas, ou *dashboards* com gráficos interativos.

Veremos um *case* real no final deste bloco de estudos, mas já adianto que essa autonomia pelo uso da IA é uma forma de multiplicarmos a produtividade. Afinal, conseguimos mostrar um protótipo e coletar *feedback* em pouco tempo.

UX *writing* para IA

Como o nosso foco desta semana é em interface visual, precisamos conhecer primeiro um conceito importante: o **UX (*user experience*, ou experiência do usuário) *writing***. Ele trata da arte de escrever textos que guiam o usuário na interação com o sistema.

Em sistemas de IA, isso é especialmente importante por alguns motivos. Vejamos:

1

O usuário não sabe o que a IA pode fazer. Diferente de uma calculadora, na qual os botões indicam as operações possíveis, uma IA conversacional pode fazer muitas coisas. O UX *writing* ajuda a comunicar essas possibilidades.

2

Erros precisam ser explicados. Quando a IA não entende uma pergunta ou não consegue respondê-la, a mensagem de erro precisa ser clara e amigável. Quer dizer: será que o usuário pediu algo que a nossa solução não providencia? Ou foi um *bug* do nosso código que devemos consertar? Ou o usuário tentou fazer algo que não deveria?

3

Expectativas precisam ser gerenciadas. Se a IA tem limitações (não acessa a internet, não sabe eventos recentes), isso precisa ser comunicado. Já escutou a expressão “o combinado não sai caro”? Pois bem: é isso.

Portanto, ao criarmos os nossos agentes procure pensar no usuário. Por exemplo:

Situação	Exemplo Ruim	Exemplo Bom
Demora na resposta	*Error: model response timeout.*	Desculpe, estou demorando mais que o normal para responder. Pode tentar novamente?
Erro na pergunta	*Invalid input.*	Não entendi sua pergunta. Tente reformular de outra forma, ou pergunte algo como “Quais são os produtos mais vendidos?”

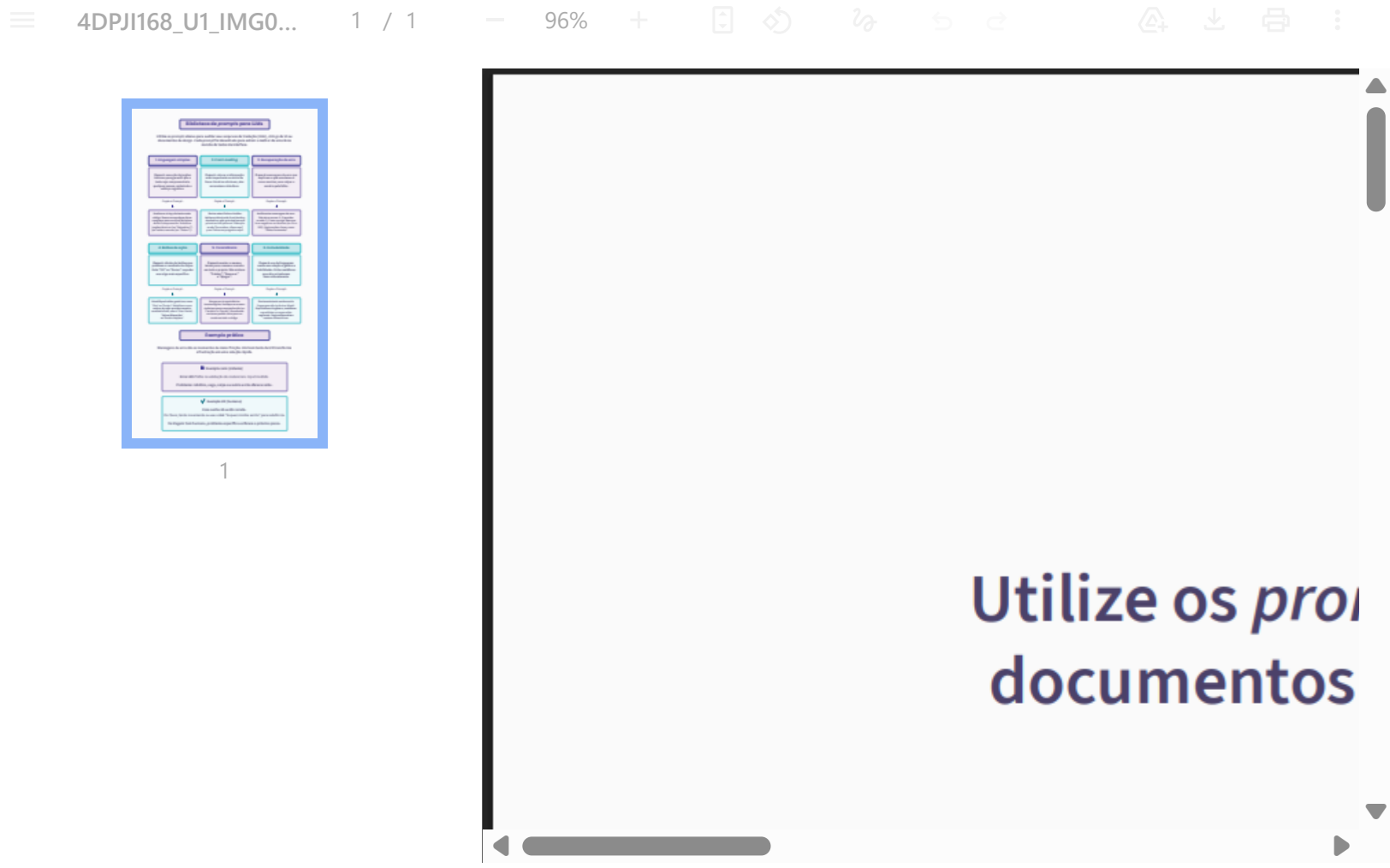
Situação	Exemplo Ruim	Exemplo Bom
Processando uma solicitação do usuário	Nenhum *feedback*.	Estou analisando a sua pergunta. Aguarde...

O autor.

Abaixo vou disponibilizar um infográfico que apresenta uma **biblioteca prática de UX *writing* aplicada ao uso de LLMs**, mostrando como a inteligência artificial pode apoiar a revisão e a melhoria de textos de interface.

Observe os **princípios essenciais** — como linguagem simples, *front-loading*, recuperação de erros, clareza em botões, consistência e inclusividade — cada um acompanhado de ***prompts* prontos para uso**, pensados para auditar e refinar microtextos de produtos digitais.

O infográfico também traz um **exemplo prático comparativo**, evidenciando a diferença entre uma mensagem técnica de sistema e uma mensagem orientada à experiência do usuário, reforçando a importância do tom humano, da clareza e da orientação à ação no *design* de interfaces.



Ao auditar seu projeto, busque converter as mensagens de sistema (vermelho) em mensagens de experiência (verde). Isso reduz o abandono de formulários e aumenta a confiança do usuário no seu produto.

#FICADICA

Quando estiver usando IA para programar, o que acha de pedir a ela para revisar o seu código e introduzir conceitos de UX writing no seu produto?

Latência percebida X Latência real

E, antes de começarmos a codar, preciso abordar outro conceito fundamental de UX: a diferença entre latência real e latência percebida.

Imagine duas situações:

Situação A: você faz uma pergunta para a IA. A tela fica completamente branca por cinco segundos. Depois, a resposta aparece de uma vez.

Situação B: Você faz a mesma pergunta. Imediatamente aparece "Pensando..." com uma animação. Depois de um segundo, o texto começa a aparecer letra por letra, como se alguém estivesse digitando. Após cinco segundos, a resposta está completa.

Nos dois casos, a latência real foi de cinco segundos. Mas qual situação 'parece' mais rápida? ⏳

A situação B, com certeza!
Pesquisas de UX nos confirmam:

- Que três segundos de tela branca parecem uma eternidade.
- Que um *feedback* visual imediato reduz a sensação de espera.
- Que animações de progresso aumentam a tolerância do usuário.
- E que o efeito "máquina de escrever" cria engajamento com o usuário.

Logo, seria legal se as nossas soluções tivessem essa sensação de maior velocidade implementada, não é?

De fato, é por isso que o Gemini, Claude, ChatGPT e outros mostram as respostas aparecendo *token* por *token*, em vez de esperar a resposta completa.

CURIOSIDADE

Estudos da Nielsen Norman Group mostram que usuários abandonam uma página se ela não responde em até 10 segundos. Com feedback visual apropriado, essa tolerância pode aumentar para 30 segundos ou mais.

Introdução ao Streamlit

Agora vamos à prática!

O **Streamlit** é o *framework* mais popular para criar aplicações de dados e IA em Python. Ele foi projetado para que cientistas de dados e engenheiros de ML possam criar interfaces web sem precisar aprender HTML, CSS ou JavaScript.

A filosofia do Streamlit é simples: ela usa a linguagem de programação Python para tudo. Ou seja, criamos um código em Python e o Streamlit transforma isto em uma aplicação web. 👌

Vamos criar a nossa primeira aplicação com o Streamlit?

O foco está na integração com o que fizemos na semana passada, bem como a introdução de alguns conceitos fundamentais como, por exemplo: configuração da página; estado da sessão (session state); a entrada e saída de mensagens trocadas entre uma IA e o usuário; streamings e call-backs.

Vamos em frente? Então aperta o play!

Para visualizar o vídeo, aponte a câmera do seu celular

Uma alternativa ao Streamlit

O **Gradio** é outra ferramenta popular para criar interfaces de ML e IA.

Enquanto o Streamlit é mais generalista (serve para qualquer app de dados), o Gradio é especializado em interfaces de *machine learning.*

A grande vantagem do Gradio para *chatbots*: você pode criar uma interface de *chat* completa em menos linhas de código. O que acha de vermos isso juntos?

Vimos sobre **Streamlit** e se fizéssemos o mesmo projeto com o Gradio? Veremos juntos um exemplo disso.

Para visualizar o vídeo, aponte a câmera do seu celular

#FICADICA

O Gradio tem integração nativa com o Hugging Face Hub. Isso significa que você pode fazer deploy do seu chatbot com um clique usando demo.launch(share=True).

Comparativo: Streamlit *vs.* Gradio

Agora, pensemos na vida real: qual dos dois deveríamos usar?

Simples! Isso depende do seu caso de uso.

Vejamos a tabela comparativa abaixo como um exemplo.

Característica	Streamlit	Gradio
Curva de aprendizado	Moderada	Muito baixa

Característica	Streamlit	Gradio
Flexibilidade	Alta (qualquer tipo de app)	Média (focado em ML)
Interface de *chat*	15-30 linhas	3-5 linhas
Customização visual	Ampla	Limitada
Deploy	Streamlit Cloud	Hugging Face Spaces
Melhor para…	*Dashboards*, *apps* complexos	Demos de ML, *chatbots*

O autor.

EM OUTRAS PALAVRAS...

Use Streamlit quando:

- Precisa de *dashboards* com gráficos e tabelas.
- Quer customização visual avançada.
- Está construindo um *app* mais complexo com múltiplas páginas.
- Precisa de controle fino sobre o *layout*.

Use Gradio quando:

- Quer uma demo rápida de um modelo de ML.
- Precisa de interface de *chat* simples.
- Quer fazer *share* instantâneo para outras pessoas testarem.
- Prioriza velocidade de desenvolvimento.

Para fechar nosso estudo de hoje, vamos conectar ideias ⬇️

Neste videocase, veremos um caso real de uso de uma IA para a melhoria da produtividade no Nubank.

Para visualizar o vídeo, aponte a câmera do seu celular

© PUCPR - Todos os direitos reservados